Universidad San Carlos de Guatemala

Centro universitario de Occidente

División de Ciencias de la ingeniería

Ingeniería en Ciencias y Sistemas

Estructura De Datos

Ing. Mario Ramirez Tobar

TDA'S
PROYECTO FINAL EDD

Cristian Alejandro Roldán López
Carné: 202147280

Quetzaltenango 04 de mayo de 2026

1. Introduccion

Un Tipo Abstracto de Dato (TAD) define un tipo de información por sus **operaciones** y **comportamiento**, independientemente de la implementación interna. El sistema implementa ocho TADs desde cero sin librerías de colecciones de Java, cada uno asignado al criterio de búsqueda u operación que mejor optimiza.

TAD	Estructura interna	Uso principal	Criterio de busqueda
Lista<T>	Lista enlazada simple	Inventario por sucursal	Recorrido / O(n)
Cola<T>	Lista enlazada FIFO	Despacho: ingreso/traspaso/salida	FIFO — O(1)
Pila<T>	Lista enlazada LIFO	Rollback de operaciones	LIFO — O(1)
ArbolAVL<T>	Arbol balanceado	Indice global por nombre	Nombre — O(log n)
ArbolB<K>	Arbol B orden T=4	Indice por fecha caducidad	Rango fechas — O(log n+k)
ArbolBPlus<K>	B+ hojas enlazadas	Indice por categoria	Categoria — O(log n+k)
TablaHash	Hash + chaining	Codigo de barras	Exacta — O(1) prom.
Grafo	Lista de adyacencia	Red sucursales + Dijkstra	Ruta optima — O(V^2)

2. TAD Lista — Inventario por Sucursal

Cada sucursal administra su inventario con una **ListaEnlazada<Productos>** propia. La lista enlazada simple permite inserción y eliminación dinámica sin reubicar elementos, ideal para inventarios de tamaño variable que cambian frecuentemente por transferencias.

Atributo	Descripción
Dominio	Secuencia ordenada de elementos T (Productos)
Operaciones	agregar(T), eliminar(T), buscar(T): T, getIndice(int): T, size(): int, isEmpty(): bool, toList()
Pre agregar	El elemento no debe ser null
Post agregar	size() aumenta en 1; el elemento es recuperable por getIndice()
Pre eliminar	!isEmpty(); el elemento existe en la lista
Post eliminar	size() disminuye en 1; buscar() retorna null para ese elemento
Invariante	size() >= 0; orden de insercion preservado; no existen nodos con referencias invalidas
Nodo interno	Nodo<T> { T dato; Nodo<T> siguiente; } con referencias head y tail

Operación	Tiempo	Espacio	Nota
agregar() al final	O(1)	O(1)	Con referencia tail directa
agregar() por pos	O(n)	O(1)	Recorrer hasta la posicion
eliminar()	O(n)	O(1)	Localizar nodo previo
buscar()	O(n)	O(1)	Recorrido secuencial
size()	O(1)	O(1)	Contador interno

3. TAD Cola — Sistema de Despacho

El enunciado define tres colas por sucursal: **Cola de ingreso** (productos recien llegados), **Cola de preparacion de traspaso** (solo nodos intermedios) y **Cola de salida** (productos listos para despachar). La disciplina FIFO modela el comportamiento real de un centro de distribucion.

Atributo	Descripcion
Dominio	Secuencia FIFO de elementos T
Operaciones	encolar(T), desencolar(): T, peek(): T, size(): int, isEmpty(): bool
Pre encolar	El elemento no debe ser null
Post encolar	size() aumenta en 1; el elemento sera el ultimo en salir
Pre desencolar	!isEmpty()

Post desencolar	size() disminuye en 1; retorna el elemento mas antiguo
Invariante FIFO	Primero en entrar = primero en salir
Implementacion	Lista enlazada con head (frente) y tail (final): encolar en tail, desencolar de head

Cola	Aplica en	Entrada	Salida hacia
colaIngreso	Todos los nodos	Producto que llega	Inventario (destino) o colaTraspaso (intermedio)
colaTraspaso	Solo intermedios	Desde colaIngreso	colaSalida
colaSalida	Origen e intermedios	Desde colaTraspaso	Siguiente sucursal de la ruta

4. TAD Pila — Rollback de Operaciones

El enunciado requiere pilas para control de cambios y operaciones de deshacer (rollback) de registros erróneos. La pila guarda un snapshot de cada operación sobre el inventario; si una inserción falla en alguna estructura durante la carga CSV, el rollback deshace las inserciones exitosas previas para mantener la consistencia entre todas las estructuras.

Atributo	Descripcion
Dominio	Secuencia LIFO de elementos T (OperacionRollback)
Operaciones	push(T), pop(): T, peek(): T, size(): int, isEmpty(): bool
Pre push	El elemento no debe ser null
Post push	peek() retorna el elemento recien insertado; size() aumenta en 1
Pre pop/peek	!isEmpty()
Post pop	size() disminuye en 1; retorna el elemento mas reciente
Invariante LIFO	El ultimo elemento en entrar es el primero en salir
Uso en CSV	Insercion atomizada: si falla en alguna estructura, pop() cada estructura exitosa

5. TAD ArbolAVL — Busqueda por Nombre

El enunciado requiere búsqueda por nombre con búsqueda binaria en árbol AVL. El AVL mantiene el índice global de productos ordenados por nombre con altura O(log n) garantizada por rotaciones automáticas, evitando la degeneración a O(n) del BST no balanceado ante inserciones ordenadas (ej. productos en orden alfabético en el CSV).

Atributo	Descripción
Dominio	Conjunto ordenado de T (comparación por nombre de producto)
Operaciones	insert(T), delete(T), search(String): T, inOrder(): ListaEnlazada<T>, getHeight(): int, isEmpty(): bool
Pre insert	Elemento no null; clave unica
Post insert	search() recupera el elemento; arbol permanece balanceado (|FB| <= 1)
Pre search	Clave no null
Post search	Retorna T si existe, null si no
Invariante AVL	|altura(izq) - altura(der)| <= 1 para todo nodo; resuelta con LL, RR, LR, RL
Nodo interno	NodoAVL<T> { T producto; NodoAVL izq, der; int altura; }

Operación	Promedio	Peor caso	Nota
insert()	O(log n)	O(log n)	Rebalanceo con rotaciones post-insercion
search()	O(log n)	O(log n)	Garantizado por invariante AVL
delete()	O(log n)	O(log n)	Rotaciones + rebalanceo ascendente
inOrder()	O(n)	O(n)	Recorrido completo en orden alfabetico
getHeight()	O(1)	O(1)	Almacenada en cada nodo

6. TAD ArbolB — Busqueda por Fecha de Caducidad

El enunciado requiere busqueda por rango de fechas. El Arbol B de orden T=4 indexa productos por fecha de caducidad (formato ISO YYYY-MM-DD). Su estructura de multiples claves por nodo reduce la altura del arbol y lo hace eficiente para consultas de rango como 'productos que vencen entre 2026-01-01 y 2026-06-30'.

Atributo	Descripcion
Dominio	Conjunto ordenado de claves K (String fecha ISO YYYY-MM-DD)
Operaciones	insert (K), search(K): bool, searchRange(K desde, K hasta): ListaEnlazada<K>, isEmpty(): bool, getRaiz(): NodoB
Pre insert	K no null; comparable con las existentes
Post insert	search(K) retorna true; arbol mantiene propiedades B
Pre searchRange	desde <= hasta (comparacion lexicografica de fechas ISO es correcta)
Post searchRange	Retorna todas K tal que desde <= K <= hasta
Invariante B	Todo nodo no raiz tiene entre ceil(T/2) y T claves; todas las hojas al mismo nivel
Orden T=4	Minimo 2 claves, maximo 7 claves por nodo

Operacion	Complejidad	Nota
insert(K)	O(log_T n)	Splits ascendentes si el nodo esta lleno
search(K)	O(log_T n)	Busqueda binaria dentro de cada nodo
searchRange	O(log n + k)	k = elementos en el rango consultado
Altura max	O(log_T n)	Mucho menor que BST: base T=4 en lugar de 2

7. TAD ArbolBPlus — Busqueda por Categoria

El enunciado requiere busqueda por categoria con recorrido en hojas enlazadas del Arbol B+. A diferencia del B, el B+ mantiene TODAS las claves en las hojas enlazadas, permitiendo recuperar todos los productos de una categoria recorriendo solo las hojas en O(k) luego del O(log n) inicial.

Atributo	Descripcion
Dominio	Claves K (categoria) en hojas enlazadas; nodos internos son guias de enrutamiento
Operaciones	insert(K), search(K): bool, searchCategory(K cat): ListaEnlazada<K>, getRaiz(): NodoBPlus, isEmpty(): bool
Pre insert	K no null
Post insert	La clave existe en exactamente una hoja; enlaces entre hojas se mantienen
Pre searchCategory	cat no null
Post searchCategory	Retorna todas las claves de la categoria cat recorriendo hojas enlazadas
Invariante B+	Todas las claves en hojas; nodos internos NO almacenan datos reales; hojas enlazadas en orden
Diferencia vs B	Hoja tiene puntero siguiente al siguiente nodo hoja; nodo interno: solo claves de enrutamiento

Operacion	Complejidad	Ventaja vs Arbol B
insert(K)	O(log n)	Splits solo en hojas — propagan al padre
search(K)	O(log n)	Siempre termina en una hoja
searchCategory(K)	O(log n + k)	k adicional por recorrido de hojas enlazadas — sin backtracking
Recorrido completo	O(n)	Solo recorrer la lista de hojas

8. TAD TablaHash — Busqueda por Codigo de Barras

El enunciado requiere busqueda hash por codigo de barras (exactamente 10 caracteres). La Tabla Hash proporciona O(1) promedio. Usa encadenamiento con **ListaEnlazada propia** por bucket (sin java.util). La capacidad inicial es 101 (numero primo) para minimizar colisiones en codigos alfanumericos de longitud fija.

Atributo	Descripcion
Dominio	Mapeo String barcode (10 chars) -> Productos
Operacion	arch(String): Productos, delete(String): bool, size(): int, getCapacidad(): int, factorCarga(): double, tam nioBucket(int)
Pre insert	producto.barcode tiene exactamente 10 chars; no existe duplicado
Post insert	search(barcode) retorna el producto; size() aumenta en 1
Pre delete	barcode no null
Post delete	search(barcode) retorna null; size() disminuye en 1
Invariante	Todo producto insertado es localizable por search(); factorCarga() < 0.75
Funcion hash	h(b) = (suma barcode[i] * 31^i) mod capacidad — base prima 31 para codigos alfanumericos

Operacion	Promedio	Peor caso	Nota
insert()	O(1)	O(n)	Peor caso: todos en un bucket
search()	O(1)	O(n)	Depende del factor de carga
delete()	O(1)	O(n)	Localizar y remover de ListaEnlazada del bucket
factorCarga	O(1)	O(1)	size / capacidad

9. TAD Grafo — Red de Sucursales y Dijkstra

El enunciado requiere un grafo ponderado donde cada nodo es una sucursal y cada arista contiene dos pesos (tiempo en minutos y costo en quetzales). El usuario elige el criterio y el algoritmo de Dijkstra calcula la ruta optima. Se implementa con lista de adyacencia para grafos dispersos (E << V^2).

Atributo	Descripcion
Dominio	Grafo no dirigido ponderado G=(V,E): V=sucursales, E=conexiones con peso dual
Operacion	agregarArista(orig,dest,tiempo,costo), dijkstra(orig,dest,usarTiempo): int[], getAristas(nodo): NodoArista
Pre agregarArista	Ambos nodos existen; tiempo > 0 y costo > 0
Post dijkstra	Retorna int[] con IDs de sucursales formando la ruta optima segun el criterio
Pre dijkstra	Nodos origen y destino existen; grafo conexo para ese par
Invariante	No dirigido: si existe arista(u,v) existe arista(v,u) con los mismos pesos
Sin aristas neg.	Garantizado: tiempos y costos reales son siempre positivos — Dijkstra aplica
Representacion	Lista de adyacencia: NodoArista[] (arreglo de listas enlazadas de aristas)
NodoArista	NodoArista { int destino; double tiempo; double costo; NodoArista siguiente; }

Operacion	Tiempo	Espacio	Nota
agregarSucursal()	O(1)	O(1)	Registrar ID
agregarArista()	O(1)	O(1)	Insertar en lista de adyacencia
dijkstra() actual	O(V^2)	O(V)	V=12 sucursales: 144 ops — muy rapido
dijkstra() optimo	O((V+E)log V)	O(V+E)	Con MinHeap para redes grandes
getAristas()	O(1)	O(1)	Acceso directo al head de la lista

10. Comparativa General de TADs

Resumen de las ocho estructuras implementadas con sus complejidades y roles en el sistema segun el enunciado de la Fase 2.

TAD	Busqueda	Insercion	Eliminacion	Espacio	Uso en el sistema
Lista<T>	O(n)	O(1)	O(n)	O(n)	Inventario por sucursal
Cola<T>	N/A	O(1)	O(1)	O(n)	Ingreso / traspaso / salida
Pila<T>	N/A	O(1)	O(1)	O(n)	Rollback CSV
ArbolAVL<T>	O(log n)	O(log n)	O(log n)	O(n)	Busqueda por nombre
ArbolB<K>	O(log n)	O(log n)	O(log n)	O(n)	Fechas (rango)
ArbolBPlus<K>	O(log n)	O(log n)	O(log n)	O(n)	Categoria (hojas)
TablaHash	O(1)*	O(1)*	O(1)*	O(n)	Codigo de barras
Grafo+Dijkstra	O((V+E)logV)	O(1)	O(E)	O(V+E)	Red y rutas

(*) O(1) promedio Hash. Peor caso O(n) cuando todas las claves colisionan en un bucket (evitado con funcion hash de base prima y rehashing a factorCarga >= 0.75).

Inserción atomizada	Al agregar un producto se inserta en TODAS las estructuras: AVL + Hash + Arbol B + Arbol B+ + Lista de sucursal. Si falla en alguna, la Pila de rollback deshace las inserciones exitosas previas. Esto cumple el requisito del enunciado de 'insercion atomizada con rollback coherente'.